TP2 – Product Flavors (Partie A)

Développement Mobile – BTS

Objectif de la partie A

Dans cette partie, l'objectif est de comprendre et mettre en place les **Product Flavors** afin de générer **deux applications différentes** à partir d'un seul projet Android :

— une application **Student**

— une application **Admin**

À la fin de cette partie, l'étudiant sera capable de :

— créer et configurer des Product Flavors

— comprendre le rôle de Gradle dans la génération des variantes

— lancer deux applications différentes sur le téléphone

— identifier le flavor actif dans le code Java

Pré-requis

— TP1a et TP1b fonctionnels

— Android Studio installé

— Connaissance de base des Activities et de Logcat

1 1. Notion de Product Flavor

Un **Product Flavor** permet de créer plusieurs versions d'une même application à partir d'un seul projet.

Exemples concrets :

— Application Étudiant / Application Admin

— Version Gratuite / Version Payante

— Version Test / Version Production

Dans ce TP, nous allons créer deux flavors :

— **student**

— **admin**

Chaque flavor générera une application différente.

2 2. Configuration des Flavors dans Gradle

Étape 1 : Ouvrir le bon fichier Gradle

L'étudiant doit ouvrir le fichier suivant :

`Gradle Scripts → build.gradle.kts (Module : app)`

Attention :

— Il faut modifier le Gradle du module `app`

— Ne pas modifier le Gradle du projet

Étape 2 : Activer les fonctionnalités nécessaires

Dans le bloc `android` , ajouter :

```
buildFeatures {
viewBinding = true
buildConfig = true
}
```

Explication :

- `viewBinding` : facilite l'accès aux vues (utilisé plus tard)
- `buildConfig` : permet à Android de générer la classe `BuildConfig`

Important : Sans `buildConfig = true`, la variable `BuildConfig.FLAVOR` ne sera pas disponible.

Étape 3 : Définir les flavors

Toujours dans le bloc `android` , ajouter :

```
flavorDimensions += "role"

productFlavors {
create("student") {
dimension = "role"
applicationIdSuffix = ".student"
versionNameSuffix = "-student"
}
create("admin") {
dimension = "role"
applicationIdSuffix = ".admin"
versionNameSuffix = "-admin"
}
}
```

Explication ligne par ligne :

- `flavorDimensions` : axe de variation (ici : rôle de l'utilisateur)
- `student / admin` : noms des flavors
- `applicationIdSuffix` : permet d'installer les deux apps sur le même téléphone
- `versionNameSuffix` : suffixe visible dans la version de l'application

Après modification, cliquer sur **Sync Now**.

3 3. Sélection et lancement des flavors

Étape 1 : Ouvrir Build Variants

Dans Android Studio :

- View → Tool Windows → Build Variants

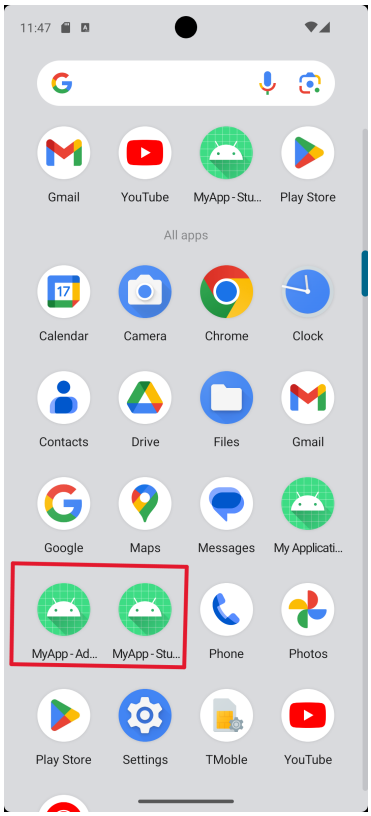

FIGURE 1 – Nom des deux applications Admin et Student sur le téléphone

Étape 2 : Choisir une variante

Dans le module `app`, choisir successivement :

- `studentDebug`
- `adminDebug`

Lancer l'application après chaque sélection.
Résultat attendu :

- Deux applications installées sur le téléphone
- Deux noms différents visibles

4 4. Adapter l'affichage selon le flavor

Dans `MainActivity.java`, écrire le code suivant :

```
String flavor = BuildConfig.FLAVOR;

if ("admin".equals(flavor)) {
Log.d(LOG_TAG, "Mode ADMIN actif");
tvWelcome.setText("Bienvenue Admin");
} else if ("student".equals(flavor)) {
Log.d(LOG_TAG, "Mode STUDENT actif");
tvWelcome.setText("Bienvenue Étudiant");
} else {
Log.d(LOG_TAG, "Flavor inconnu : " + flavor);
}
```

Explication :

— `BuildConfig.FLAVOR` contient le flavor actif

— Le texte affiché change selon l'application lancée

— Le Logcat permet de vérifier le mode actif

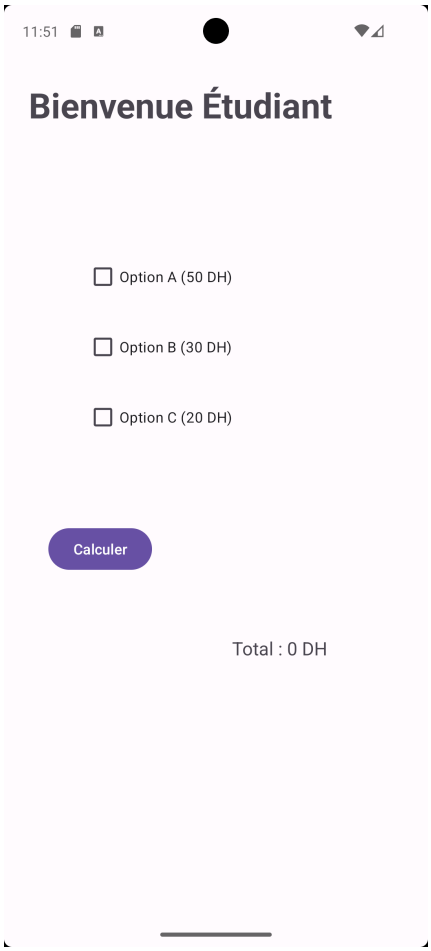

FIGURE 2 – Message *Bienvenue Étudiant* affiché

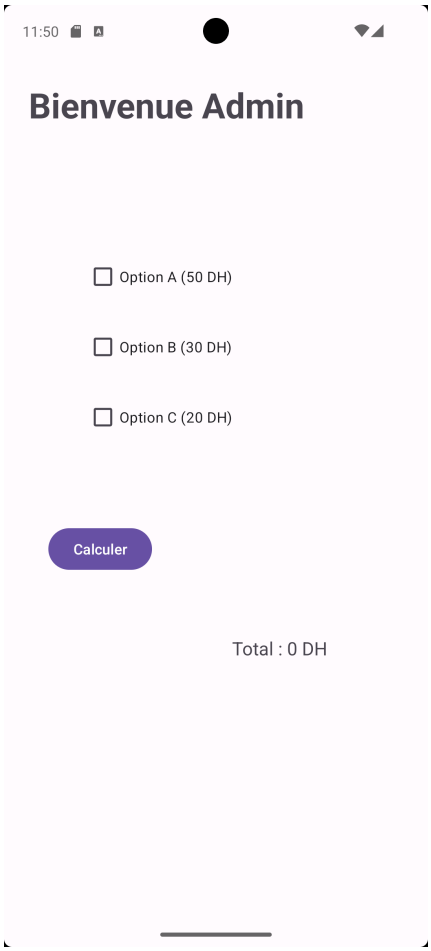

FIGURE 3 – Message *Bienvenue Admin* affiché

5 Conclusion de la partie A

Dans cette partie, nous avons appris à :

— créer deux Product Flavors

— générer deux applications différentes

— identifier le flavor actif dans le code

La partie B consistera à adapter la navigation et les interfaces selon le rôle de l'utilisateur.